

AULA 05

Funções e procedimentos armazenados em PL/pgSQL

Banco de Dados II — Prof. Alysson

12/09/2026

Recapitulando as Aulas 01 a 04

- **Aula 01:** dados, SGBD, modelo relacional e PostgreSQL
- **Aula 02:** normalização, `INSERT`, `UPDATE`, `DELETE` e integridade
- **Aula 03:** junções, subconsultas e operações de conjunto
- **Aula 04:** PostgreSQL em contêiner, cliente-servidor e organização física

Já sabemos **consultar** o banco e **executar** o servidor.

Hoje vamos aprender

- Por que uma **regra de negócio** pode morar dentro do banco
- Como é feita uma **função** em PL/pgSQL
- Como declarar **parâmetros** e **variáveis**
- Como devolver **um valor** ou **uma tabela**
- O que é um **procedimento** e quando usá-lo
- Como criar, testar e apagar rotinas no PostgreSQL

Pergunta de partida

Uma loja precisa saber **quanto custa o pedido nº 1**.

```
SELECT SUM(quantidade * preco_unitario)
FROM item_pedido
WHERE pedido_id = 1;
```

A consulta funciona. Mas o site, o aplicativo e o relatório precisam da **mesma** conta.

Quem deve guardar essa regra?

Parte 1

Por que programar dentro do banco?

O problema da regra repetida

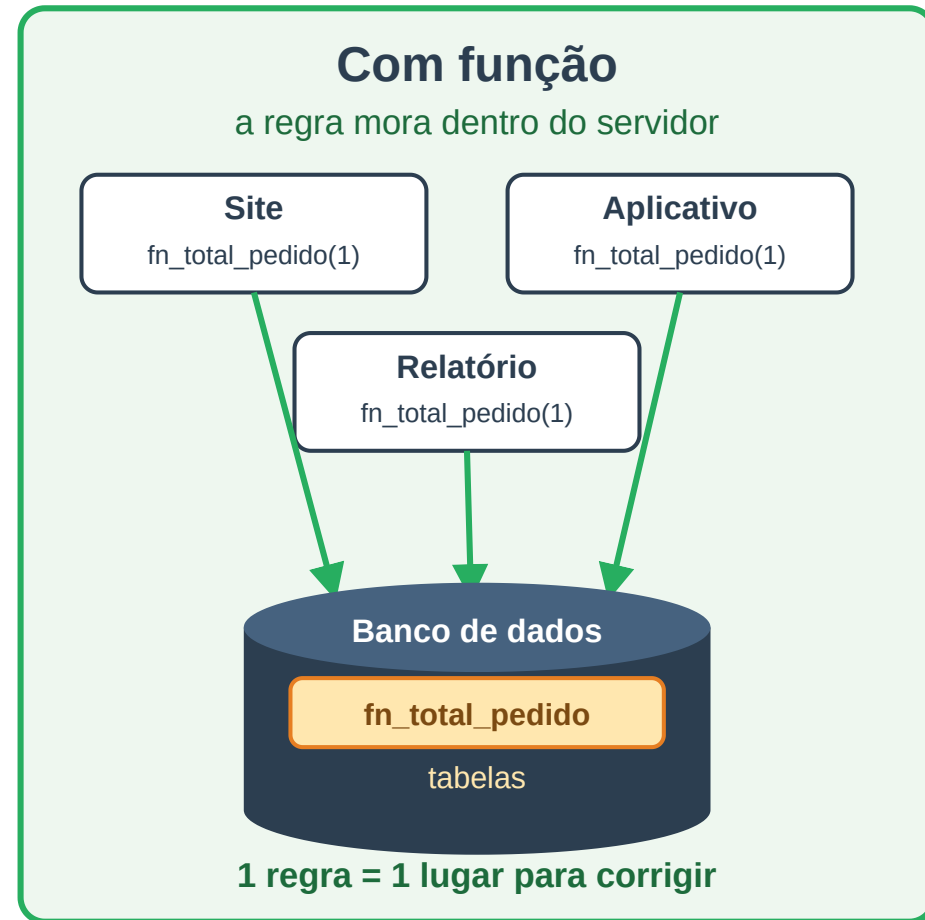
```
Site      → SUM(quantidade * preco_unitario)
Aplicativo → SUM(quantidade * preco_unitario)
Relatório → SUM(quantidade * preco_unitario)
```

A empresa decide dar 5% de desconto para pagamento à vista.

Quantos programas precisam ser alterados? Três.

E se um deles for esquecido? O sistema passa a dar dois valores diferentes para o mesmo pedido.

A mesma regra, dois lugares possíveis



O que ganhamos

- **Uma regra só:** corrigiu no banco, corrigiu para todos
- **Reúso:** site, aplicativo e relatório chamam o mesmo nome
- **Menos tráfego:** o cálculo acontece onde os dados estão
- **Padronização:** o nome da rotina vira o vocabulário da equipe
- **Segurança:** é possível liberar a rotina sem liberar a tabela

O que exige cuidado

- Código dentro do banco também precisa de **versionamento**
- É preciso **testar**: um erro afeta todos os sistemas de uma vez
- Regras demais no banco dificultam a **portabilidade** para outro SGBD
- Nem toda regra pertence ao banco: interface e fluxo de tela ficam na aplicação

Critério prático: se a regra é sobre **os dados**, ela costuma caber no banco.

Onde isso aparece no dia a dia

Situação	Rotina no banco
Somar o total de um pedido	função de cálculo
Aplicar desconto padronizado	função de cálculo
Listar os pedidos de um cliente	função que devolve tabela
Dar baixa no estoque	procedimento
Fechar o mês e arquivar registros	procedimento

Parte 2

A linguagem PL/pgSQL

O que é PL/pgSQL

- É a **linguagem de programação** que acompanha o PostgreSQL
- Junta comandos SQL com **variáveis**, **decisões** e **repetições**
- O código fica **guardado dentro do banco** e é executado pelo servidor
- **Rotina** é o nome geral para **função** e **procedimento**

```
rotina = nome + parâmetros + corpo
```

Ideia central: SQL diz o *que* queremos; PL/pgSQL organiza a *sequência* de passos.

Duas linguagens possíveis

<code>LANGUAGE sql</code>	<code>LANGUAGE plpgsql</code>
corpo é uma consulta	corpo é um bloco de passos
não tem variáveis	tem variáveis
não tem decisão nem repetição	tem <code>IF</code> , <code>LOOP</code> e exceções
ótima para cálculos simples	necessária para lógica

Comece pela mais simples; use PL/pgSQL quando precisar de mais.

A mesma regra em `LANGUAGE sql`

```
CREATE OR REPLACE FUNCTION fn_desconto_sql(  
    p_preco      numeric,  
    p_percentual numeric  
)  
RETURNS numeric  
LANGUAGE sql  
AS $$  
    SELECT round(p_preco - p_preco * p_percentual / 100, 2);  
$$;
```

Uma consulta apenas. O resultado dela é o resultado da função.

A mesma regra em `LANGUAGE plpgsql`

```
CREATE OR REPLACE FUNCTION fn_desconto(  
    p_preco      numeric,  
    p_percentual numeric DEFAULT 10  
)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_valor numeric;  
BEGIN  
    v_valor := p_preco - p_preco * p_percentual / 100;  
    RETURN round(v_valor, 2);  
END;  
$$;
```

Mais linhas, porém com espaço para variáveis e decisões.

Anatomia de uma função

```
CREATE OR REPLACE FUNCTION
    fn_total_pedido(p_pedido_id integer)

RETURNS numeric

LANGUAGE plpgsql

AS $$

DECLARE
    v_total numeric;

BEGIN
    SELECT ... INTO v_total ...;

    RETURN v_total;

END;

$$;
```

1. nome e parâmetros

2. tipo de dado devolvido

3. linguagem do corpo

4. os cifrões abrem o corpo

5. variáveis da rotina

6. o que a função faz

7. fecha o corpo e o comando

Peças 1 e 2 — cabeçalho e retorno

```
CREATE OR REPLACE FUNCTION fn_total_pedido(p_pedido_id integer)
RETURNS numeric
```

- `OR REPLACE` **substitui** a versão anterior, se já existir
- `fn_total_pedido` é o nome pelo qual todos vão chamá-la
- `p_pedido_id integer` é um **parâmetro**: o dado que ela recebe
- `RETURNS` declara o tipo do resultado — pode ser um valor simples, uma **tabela** (`RETURNS TABLE`) ou `void`

Peças 3 e 4 — linguagem e cifrões

```
LANGUAGE plpgsql
AS $$
    ...corpo da rotina...
$$;
```

- `LANGUAGE` informa ao PostgreSQL **como ler o corpo**; sem ela, o comando falha
- O corpo é entregue ao servidor como **texto**
- Os cifrões (`$$`) marcam onde esse texto começa e termina
- Assim podemos usar **aspas simples** dentro do corpo sem confusão

Peças 5 e 6 — declarações e bloco executável

```
DECLARE
    v_total numeric;
BEGIN
    SELECT COALESCE(SUM(quantidade * preco_unitario), 0)
        INTO v_total
        FROM item_pedido
        WHERE pedido_id = p_pedido_id;

    RETURN v_total;
END;
```

- `DECLARE` é opcional: sem variáveis, comece direto no `BEGIN`
- Variável **não** é parâmetro: o parâmetro vem de fora, a variável nasce dentro
- `RETURN` devolve o resultado e **encerra** a função

Convenção de nomes da disciplina

Prefixo	Significado	Exemplo
p_	parâmetro recebido	p_pedido_id
v_	variável interna	v_total
fn_	função	fn_total_pedido
sp_	procedimento	sp_baixar_estoque

Isso evita o erro mais comum: **variável com o mesmo nome de uma coluna**.

Guardar valores em variáveis

Atribuição direta:

```
v_valor := p_preco * 0.9;
```

Resultado de uma consulta:

```
SELECT COALESCE(SUM(quantidade * preco_unitario), 0)
      INTO v_total
      FROM item_pedido
      WHERE pedido_id = p_pedido_id;
```

`:=` atribui. `INTO` guarda o resultado da consulta na variável.

Parâmetro com valor-padrão

```
CREATE OR REPLACE FUNCTION fn_desconto(  
    p_preco      numeric,  
    p_percentual numeric DEFAULT 10  
) RETURNS numeric ...
```

```
SELECT fn_desconto(120.00, 25);    -- 90.00  
SELECT fn_desconto(120.00);        -- 108.00
```

Quando o parâmetro é omitido, o banco usa o valor-padrão.

Função que devolve uma tabela

```
CREATE OR REPLACE FUNCTION fn_pedidos_do_cliente(p_cliente_id integer)
RETURNS TABLE (
    pedido_id    integer,
    data_pedido  date,
    valor_total  numeric
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT p.id, p.data_pedido, fn_total_pedido(p.id)
           FROM pedido p
          WHERE p.cliente_id = p_cliente_id
          ORDER BY p.id;
END;
$$;
```

Como chamar uma função que devolve tabela

```
SELECT * FROM fn_pedidos_do_cliente(1);
```

pedido_id	data_pedido	valor_total
1	2026-09-12	275.00
2	2026-09-12	900.00

A função aparece no `FROM`, como se fosse uma tabela.

Cuidado com nomes ambíguos

- Em `RETURNS TABLE`, os nomes declarados viram **variáveis**
- Se a consulta usar o mesmo nome sem apelido, o banco reclama:

```
ERROR: column reference "data_pedido" is ambiguous
```

Solução: qualifique as colunas com o apelido da tabela.

```
SELECT p.id, p.data_pedido FROM pedido p ...
```

Parte 3

Procedimientos

Função × procedimento

FUNÇÃO

Devolve

sempre um resultado: número, texto, data ou até uma tabela

É chamada

dentro de uma consulta

```
SELECT fn_total_pedido(1);
```

Transação

não pode confirmar nem desfazer;
participa da transação de quem chamou

Uso típico: calcular e consultar

PROCEDIMENTO

Devolve

nada por padrão; pode devolver valores por parâmetros INOUT

É chamado

por um comando próprio

```
CALL sp_baixar_estoque(3, 1);
```

Transação

pode executar COMMIT e ROLLBACK dentro do próprio corpo

Uso típico: executar e alterar

Criando um procedimento (1/2)

```
CREATE OR REPLACE PROCEDURE sp_baixar_estoque(  
    p_produto_id integer,  
    p_quantidade integer  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_estoque integer;  
BEGIN  
    SELECT estoque INTO v_estoque  
    FROM produto WHERE id = p_produto_id;
```

Repare: **não existe RETURNS**.

Criando um procedimento (2/2)

```
IF v_estoque < p_quantidade THEN
    RAISE EXCEPTION 'Estoque insuficiente: há % unidades.',
        v_estoque;
END IF;

UPDATE produto SET estoque = estoque - p_quantidade
WHERE id = p_produto_id;

COMMIT;

END;
$$;
```

- O `COMMIT` confirma a alteração — algo que **só** o procedimento pode fazer.

Chamando um procedimento

```
CALL sp_baixar_estoque(3, 1);
```

- Não existe `SELECT sp_baixar_estoque(...)`
- O procedimento é chamado por um comando próprio: `CALL`
- Ele **faz** alguma coisa; não precisa devolver nada

Verificação: consulte a tabela antes e depois da chamada.

Por que só o procedimento confirma a transação

- Uma **função** é executada **dentro** do comando que a chamou
- Ela não pode encerrar uma transação que não começou
- Um **procedimento** é chamado sozinho, por `CALL`
- Por isso pode usar `COMMIT` e `ROLLBACK` no próprio corpo

Detalhe: se o `CALL` estiver dentro de um `BEGIN` aberto pelo usuário, o `COMMIT` interno é recusado.

Devolvendo valor por parâmetro

```
CREATE OR REPLACE PROCEDURE sp_abrir_pedido(  
    p_cliente_id      integer,  
    INOUT p_pedido_id integer  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO pedido (cliente_id) VALUES (p_cliente_id)  
    RETURNING id INTO p_pedido_id;  
END;  
$$;
```

```
CALL sp_abrir_pedido(2, NULL);
```

Qual escolher?

Preciso de...	Use
um valor dentro de um <code>SELECT</code>	função
uma lista de linhas	função com <code>RETURNS TABLE</code>
executar uma ação sobre os dados	procedimento
controlar <code>COMMIT</code> / <code>ROLLBACK</code>	procedimento

Na dúvida: **calculou, é função; executou, é procedimento.**

Parte 4

Laboratório no iximiuz Labs

Passo 0 — abrir o ambiente

1. Acesse <https://labs.iximiuz.com/playgrounds/BD2-AULA05-dedf2f1b>
2. Clique em **Start Playground** e aguarde a máquina subir
3. Use a aba **Term** para digitar os comandos
4. Para colar: **Ctrl + Shift + V**

O PostgreSQL 17 já vem baixado e os scripts da aula já estão em `~/lab05` .
Nada é instalado no seu computador.

Passo 1 — subir o PostgreSQL

```
podman run -d --name bd2-pg \  
-e POSTGRES_PASSWORD=bd2 \  
-e POSTGRES_DB=loja \  
docker.io/library/postgres:17
```

Confira:

```
podman ps  
podman exec bd2-pg pg_isready -U postgres
```

Pare e confira: apareceu `accepting connections` ?

Passo 2 — entrar no psql

```
podman exec -it bd2-pg psql -U postgres -d loja
```

O prompt muda para `loja=#`. Daqui em diante, tudo é SQL.

Comando	Efeito
<code>\dt</code>	lista as tabelas
<code>\df</code>	lista funções e procedimentos
<code>\sf nome</code>	mostra o código da rotina
<code>\q</code>	sai do psql

Passo 3 — criar as tabelas

```
CREATE TABLE cliente (  
    id serial PRIMARY KEY,  
    nome text NOT NULL,  
    cidade text NOT NULL);  
  
CREATE TABLE produto (  
    id serial PRIMARY KEY,  
    nome text NOT NULL,  
    preco numeric(10,2) NOT NULL CHECK (preco > 0),  
    estoque integer NOT NULL DEFAULT 0 CHECK (estoque >= 0));  
  
CREATE TABLE pedido (  
    id serial PRIMARY KEY,  
    cliente_id integer NOT NULL REFERENCES cliente (id),  
    data_pedido date NOT NULL DEFAULT CURRENT_DATE,  
    situacao text NOT NULL DEFAULT 'aberto');
```

Passo 3 — criar as tabelas (continuação)

```
CREATE TABLE item_pedido (  
    pedido_id integer NOT NULL REFERENCES pedido (id),  
    produto_id integer NOT NULL REFERENCES produto (id),  
    quantidade integer NOT NULL CHECK (quantidade > 0),  
    preco_unitario numeric(10,2) NOT NULL CHECK (preco_unitario > 0),  
    PRIMARY KEY (pedido_id, produto_id));
```

Confira com `\dt` : devem aparecer quatro tabelas.

Passo 3 — inserir os dados

```
INSERT INTO cliente (nome, cidade) VALUES
    ('Ana Souza', 'Palmas'), ('Bruno Lima', 'Porto Nacional'),
    ('Carla Dias', 'Paraíso do Tocantins');

INSERT INTO produto (nome, preco, estoque) VALUES
    ('Teclado', 120.00, 10), ('Mouse', 80.00, 15),
    ('Monitor', 900.00, 4), ('Cabo HDMI', 35.00, 25);

INSERT INTO pedido (cliente_id) VALUES (1), (1), (2);

INSERT INTO item_pedido VALUES
    (1, 1, 2, 120.00), (1, 4, 1, 35.00), (2, 3, 1, 900.00), (3, 2, 3, 80.00);
```

Passo 4 — a pergunta

Quanto custa o pedido nº 1?

Ele tem dois itens:

2 unidades de Teclado	a	120,00
1 unidade de Cabo HDMI	a	35,00

Antes de olhar o próximo slide: **qual consulta responde isso?**

Passo 4 — a resposta

```
SELECT SUM(quantidade * preco_unitario) AS total
FROM item_pedido
WHERE pedido_id = 1;
```

```
total
-----
275.00
```

Guarde esse número. No **Passo 7** essa consulta vira uma função.

Passo 5 — primeira função

Comece por uma regra mais simples, só para fixar o formato.

```
CREATE OR REPLACE FUNCTION fn_desconto_sql(  
    p_preco numeric, p_percentual numeric  
)  
RETURNS numeric  
LANGUAGE sql  
AS $$  
    SELECT round(p_preco - p_preco * p_percentual / 100, 2);  
$$;
```

```
SELECT fn_desconto_sql(120.00, 10);    -- 108.00
```

Passo 5 — usando a função em uma consulta

```
SELECT nome,  
       preco,  
       fn_desconto_sql(preco, 15) AS promocao  
FROM produto  
ORDER BY id;
```

A função entra no `SELECT` como qualquer outra expressão e é aplicada **linha a linha**.

Passo 6 — a mesma regra em PL/pgSQL

```
CREATE OR REPLACE FUNCTION fn_desconto(  
    p_preco numeric, p_percentual numeric DEFAULT 10  
)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_valor numeric;  
BEGIN  
    v_valor := p_preco - p_preco * p_percentual / 100;  
    RETURN round(v_valor, 2);  
END;  
$$;
```

```
SELECT fn_desconto(120.00, 25), fn_desconto(120.00); -- 90.00 e 108.00
```

Passo 7 — a pergunta

Queremos uma função chamada assim:

```
SELECT fn_total_pedido(1);
```

Ela precisa:

1. receber o **número do pedido**;
2. somar `quantidade * preco_unitario` dos itens;
3. devolver `0` quando o pedido não tiver itens.

Como fica o corpo dessa função?

Passo 7 — a resposta

```
CREATE OR REPLACE FUNCTION fn_total_pedido(p_pedido_id integer)
RETURNS numeric
LANGUAGE plpgsql
AS $$
DECLARE
    v_total numeric;
BEGIN
    SELECT COALESCE(SUM(quantidade * preco_unitario), 0)
        INTO v_total
        FROM item_pedido
        WHERE pedido_id = p_pedido_id;
    RETURN v_total;
END;
$$;
```

COALESCE troca o resultado vazio por zero.

Passo 7 — testando

```
SELECT fn_total_pedido(1);

SELECT id, fn_total_pedido(id) AS total
FROM pedido
ORDER BY id;
```

id	total
1	275.00
2	900.00
3	240.00

Uma regra, três pedidos, nenhuma repetição de código.

Passo 8 — devolvendo uma tabela

```
CREATE OR REPLACE FUNCTION fn_pedidos_do_cliente(p_cliente_id integer)
RETURNS TABLE (pedido_id integer, data_pedido date, valor_total numeric)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT p.id, p.data_pedido, fn_total_pedido(p.id)
           FROM pedido p
          WHERE p.cliente_id = p_cliente_id
          ORDER BY p.id;
END;
$$;
```

```
SELECT * FROM fn_pedidos_do_cliente(1);
```

Passo 9 — o procedimento (1/2)

```
CREATE OR REPLACE PROCEDURE sp_baixar_estoque(  
    p_produto_id integer, p_quantidade integer  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_estoque integer;  
BEGIN  
    SELECT estoque INTO v_estoque  
    FROM produto WHERE id = p_produto_id;
```

Continua no próximo slide — **não execute ainda.**

Passo 9 — o procedimento (2/2)

```
IF v_estoque < p_quantidade THEN
    RAISE EXCEPTION
        'Estoque insuficiente: há % unidades e foram pedidas %.',
        v_estoque, p_quantidade;
END IF;

UPDATE produto SET estoque = estoque - p_quantidade
WHERE id = p_produto_id;

COMMIT;

END;
$$;
```

Passo 9 — testando o procedimento

```
SELECT id, nome, estoque FROM produto WHERE id = 3;  
CALL sp_baixar_estoque(3, 1);  
SELECT id, nome, estoque FROM produto WHERE id = 3;
```

Resultado esperado: o estoque do monitor cai de 4 para 3 .

O CALL executou a ação e confirmou a alteração.

Passo 10 — errar de propósito

```
CALL sp_baixar_estoque(3, 500);
```

```
ERROR: Estoque insuficiente: há 3 unidades e foram pedidas 500.
```

- A rotina **parou antes** do UPDATE
- O estoque continua igual
- A regra vale para **qualquer** aplicação conectada ao banco

Passo 11 — inspecionar e manter

```
\df  
\sf fn_total_pedido
```

```
DROP FUNCTION fn_desconto_sql(numeric, numeric);
```

- `\df` lista o que existe no banco
- `\sf` mostra o código-fonte da rotina
- No `DROP`, informe também **os tipos dos parâmetros**

Encerrando o laboratório

Ao fechar o playground, o iximiuz pergunta o que fazer com o ambiente:

Opção	Efeito
Stop	guarda o estado; você retoma depois de onde parou
Destroy	apaga o ambiente definitivamente

Para continuar em casa, escolha **Stop** — e anote os comandos que importam.

Parte 5

Erros comuns e fechamento

Erros mais comuns (1/2)

Mensagem	Causa provável
no language specified	faltou a cláusula LANGUAGE
unterminated dollar-quoted string	faltou fechar \$\$;
column reference ... is ambiguous	variável com nome de coluna
function ... does not exist	nome ou tipo do parâmetro errado

Erros mais comuns (2/2)

- Esquecer o **ponto e vírgula** ao fim de cada comando interno
- Trocar `=` por `:=` na atribuição
- Chamar procedimento com `SELECT` em vez de `CALL`
- Esperar que a função devolva algo sem escrever `RETURN`
- Apagar a rotina errada por não informar os tipos no `DROP`

Dica: leia a mensagem de erro até o fim; o PostgreSQL informa a linha.

Mapa-resumo

```
CREATE FUNCTION → RETURNS → LANGUAGE → $$ corpo $$  
                                     ↙           ↘  
                               um valor      uma tabela  
                        (SELECT fn(...))  (SELECT * FROM fn(...))
```

```
CREATE PROCEDURE → CALL → executa e pode confirmar a transação
```

Para guardar

- **Rotina** = nome + parâmetros + corpo
- `LANGUAGE sql` para cálculos simples; `plpgsql` quando há lógica
- Os **cifrões** delimitam o corpo da rotina
- `p_` para parâmetro e `v_` para variável evitam ambiguidade
- **Função** devolve e é chamada em consultas; **procedimento** age e é chamado por `CALL`
- Só o procedimento pode executar `COMMIT` e `ROLLBACK`

Atividade de consolidação

- Leia o texto-base em blocos curtos
- Refaça o laboratório no iximiuz Labs ou releia os comandos anotados
- Escreva **uma função sua** sobre o esquema da loja
(sugestão: `fn_estoque_baixo(p_limite integer)`)
- Responda às **dez questões** no Google Classroom
- Turma no Classroom: <https://classroom.google.com/c/ODY5MzE1NDkyODUx>

Encerramento

**Na próxima aula: gatilhos, cursores,
estruturas de controle e tratamento de
exceções**

Dúvidas?